Notebook on

Computer Architecture

Prepared by

Shahriar Arefin

ID: 22101040

Section: (A)

Faculty

Md. Rasheduzzaman

Semester: Fall 2024

Notes from the Computer Architecture class.

Contents

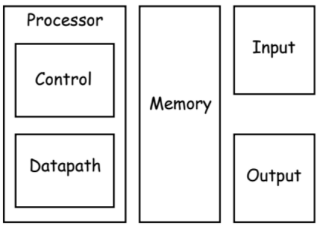
Chapter 1

Computer Architecture

1.1 Introduction to Computer Architecture

Computer architecture refers to the design and organization of a computer's components. Every computer from past to present, have a structure divided into five categories. Five classic components of a computer are —

- **Control Unit:** This component manages the operations of the computer. It directs the flow of data between the CPU and other components.
- **Memory:** This is where data and instructions are stored. It is a crucial part of the computer system that allows for the storage and retrieval of information.
- **Datapath:** The component of the processor that performs arithmetic operations e.g ALU, Registers and Bus.
- **Input:** This refers to the devices or methods through which data is entered into the computer system.
- **Output:** This refers to the devices or methods through which data is presented to the user or other systems.



1.1.1 The Role of Buses

- ***Data Bus*:** Transfers data between components.
- ***Address Bus*:** Carries the memory addresses for data retrieval.
- ***Control Bus*:** Sends control signals to manage the flow of data and instructions.

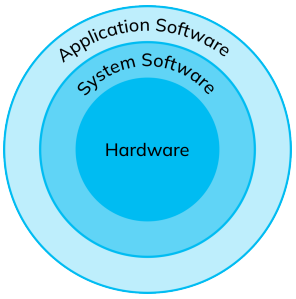
1.1.2 Types of Processor

- ***CISC (Complex Instruction Set Computer)*:**
  A processor that uses complex instructions with many addressing modes.
- ***RISC (Reduced Instruction Set Computer)*:**
  A processor with a small, simple set of instructions, making it faster for many tasks.
- ***DSP (Digital Signal Processor)*:**
  A specialized processor used for handling signal processing, often used in audio or video applications.

1.1.3 Types of Memory

- Registers: Fastest, but smallest form of memory.
- Cache: Stores frequently accessed data for quicker access.
- Main Memory (RAM): Primary memory for active tasks.
- Secondary Storage (Hard Disk, SSD): Long-term storage for data.

1.1.4 Layers of Computer

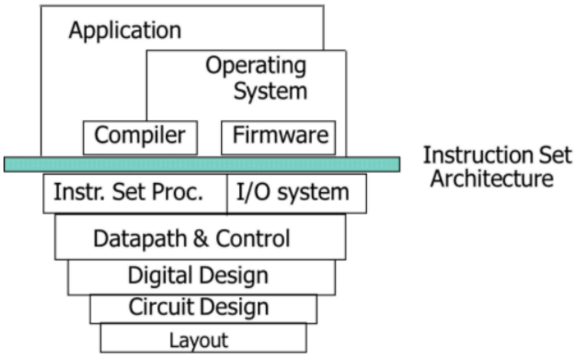


- **Hardware:** Physical components of the computer.
- **Operating System:** Software that manages hardware and software resources.
- **Applications:** Programs that perform specific tasks for users.

The layers of software may compose of Application Software and System Software. These can be organized primarily in a hierarchical fashion, with applications being the outermost ring and a variety of systems soft ware sitting between the hardware and applications software.

1.1.5 HW and SW

Both hardware and soft ware consist of hierarchical layers, with each lower layer hiding details from the level above. One key interface between the levels of abstraction is the instruction set architecture— the interface between the hardware and low-level soft ware.



Chapter 2

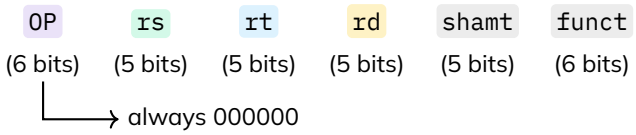
MIPS

2.1 MIPS Instructions

These are the three main instruction formats in the MIPS (Microprocessor without Interlocked Pipeline Stages) architecture: `R`, `I`, `J`

2.1.1 R - Register Type

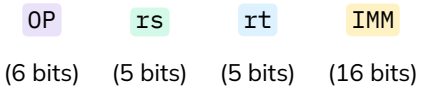
R-type instructions are used for operations that involve only registers. They have three register operands and a function code to specify the operation.



Example: `add $t0, $t1, $t2` → (Addition) `$t0 = $t1 + $t`
`sll $t0, $t1, 2` → (Shift Left) `$t0 = $t1 << 2`

2.1.2 I - Immediate Type

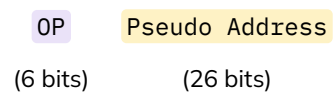
I-type instructions are used for operations that involve an immediate value (a constant) and registers. They have two register operands and a 16-bit immediate value.



Example: `lw $t0, 4($t1)` → Load word from memory at address `($t1+4)` into `$t0`

2.1.3 J - Jump Type

J-type instructions are used for jump operations, which change the program counter to a new address. They have a 26-bit target address.



Example: `j TARGET` Jump to the instruction at address TARGET
`jal FUNCTION` Jump to FUNCTION and save return address in `$ra`

2.2 Common MIPS Instruction Set

2.2.1 Arithmetic Instructions

- `add` - Add two registers
- `addi` - Add reg and immediate
- `addu` - unsinged
- `sub` - Subtract two registers
- `subi` - Sub reg and immediate
- `subu` - sub unsigned
- `mul` - Multiply two registers
- `div` - Divide two registers

2.2.2 Logical Instructions

- `and` - Bitwise AND
- `or` - Bitwise OR
- `xor` - Bitwise XOR
- `nor` - Bitwise NOR
- `not`- Bitwise Not

2.2.3 Shift Instructions

- `sll` - Shift Left Logical
- `srl` - Shift Right Logical
- `sra` - Shift Right Arithmetic

2.2.4 Memory Access Instructions

- `lw` - Load Word (from memory to a register)
- `sw` - Store Word (from a register to memory)
- `lb` - Load Byte (from memory to a register)
- `sb` - Store Byte (from a register to memory)

2.2.5 Control Flow Instructions

- `beq` - Branch if Equal
- `bne` - Branch if Not Equal
- `j` - Jump to a specified address
- `jal` - Jump and Link (used for function calls)
- `jr` - Jump Register (used for returning from a function)

2.3 MIPS Registers

Register	#Reg no.	Name	Usage	Preserved across calls?
`$zero`	0	Zero Register	Always holds `0`	Yes
`$at`	1	Assembler Temp	Reserved for assembler (pseudo-instructions)	No
`$v0-$v1`	2-3	Return Values	Used for function return values	No
`$a0-$a3`	4-7	Arguments	Used to pass function arguments	No
`$t0-$t7`	8-15	Temporary	Used for temporary values	No
`$s0-$s7`	16-23	Saved Registers	Used for saved values, preserved across calls	Yes
`$t8-$t9`	24-25	Temporary	Additional temporary registers	No
`$k0-$k1`	26-27	Kernel Reserved	Used by the OS kernel	N/A
`$gp`	28	Global Pointer	Points to global variables	Yes
`$sp`	29	Stack Pointer	Points to the top of the stack	Yes
`$fp`	30	Frame Pointer	Used for function stack frames	Yes
`$ra`	31	Return Address	Stores return address for function calls	No

✪ `Memory Address = 4 * Offset + Base Address`

2.4 MIPS Problems

#Question: Let, A = 5, B = 5, and C = 0. If A is not equal to B, subtract B from A and store the result in C. Otherwise, add A and B and store the result in C. Now write the MIPS instructions for the given problem.

***Solution*:**

```
# Load values into registers
li $t0, 5       # A = 5 ($t0)
li $t1, 5       # B = 5 ($t1)
li $t2, 0       # C = 0 ($t2)

beq $t0, $t1, SUM

sub $t2, $t0, $t1
j END

SUM:
   add $t2, $t0, $t1

END:
   jr $ra
```

```
int a = 5;
int b = 5;
int c = 0;

if (a != b) {
   c = a - b;
} else {
   c = a + b;
}
```

#Question: Let, i = 0, sum = 0, and limit = 5. Using a while loop, keep adding i to sum and incrementing i until i is no longer less than limit. Now write the MIPS instructions for this problem.

***Solution*:**

```
.data

.text
.globl main
main:
    li   $t0, 0
    li   $t1, 0
    li   $t2, 5

loop:
    beq  $t0, $t2, end
    add  $t1, $t1, $t0
    addi $t0, $t0, 1
    j    loop

end:
    jr   $ra
```

```
int i = 0;        => $t0
int sum = 0;      => $t1
int limit = 5;    => $t2

while (i < limit)
   sum = sum + i;
   i = i + 1;
}
```

Question: Instruction and Opcode/Function: `lw 100011`, `sub 101011`, `sub 100010`, `add 100000`. Analyze the following high level statement according to MIPS instruction format and write the MIPS machine code.
`A[40] = B - A[60];`

Solution:

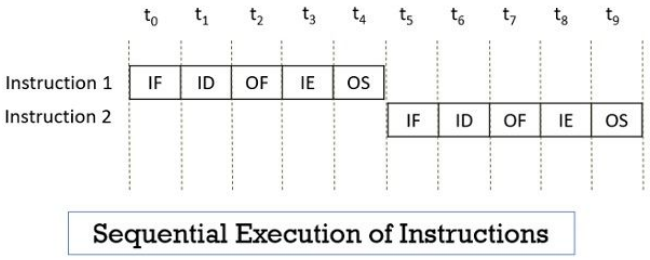
```
lw  $t0, 240($s0)           # Load A[60] into $t0
sub $t1, $s1, $t0           # Subtract B from A[60]
sw  $t1, 160($s0)           # Store result into A[40]
```

Chapter 3

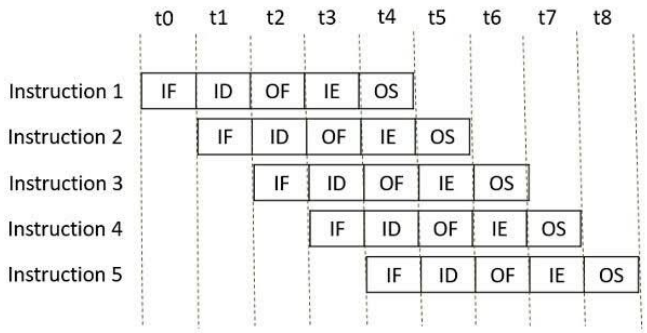
Pipelining

3.1 What is Pipelining?

Pipeline is a technique used in CPUs to execute multiple instructions simultaneously by breaking the instruction execution process into stages.



Sequential Execution of Instructions



Pipelining of 5 Instructions

Typical stages in a CPU pipeline:

- Fetch (IF) - Get the instruction from memory.
- Decode (ID) - Determine what the instruction does.
- Execute (EX) - Perform the operation (e.g., ALU computation).
- Memory Access (MEM) - Read/write data from/to memory.
- Write Back (WB) - Store the result back to a register.

Benefits of Pipelining:

- Increased throughput: Multiple instructions are in progress at the same time — so more instructions complete per second.
- Efficient use of CPU: All parts of the CPU stay busy doing specific tasks in parallel.
- Faster instruction processing for workloads: Especially when executing many instructions — like in loops or large programs.

Chapter 4

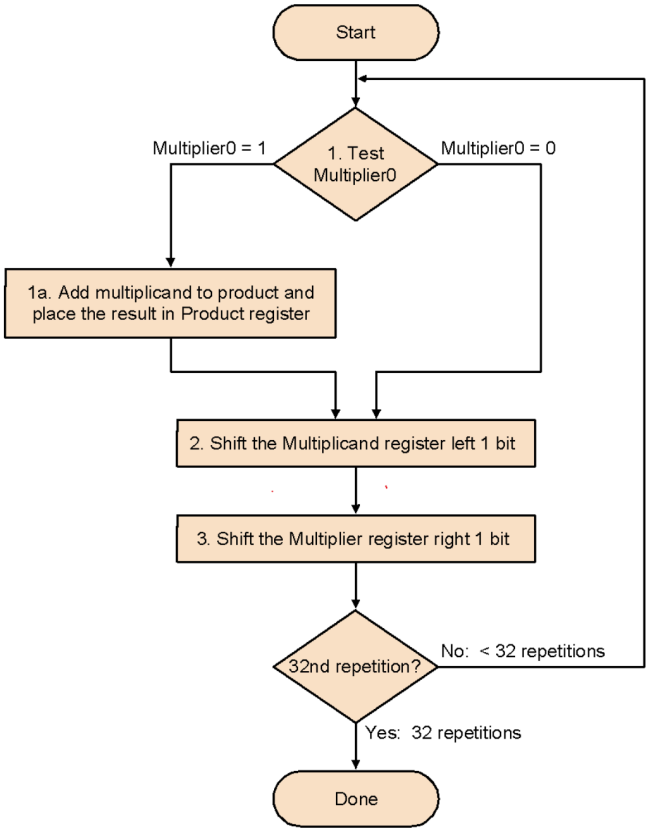
Computer Arithmetic

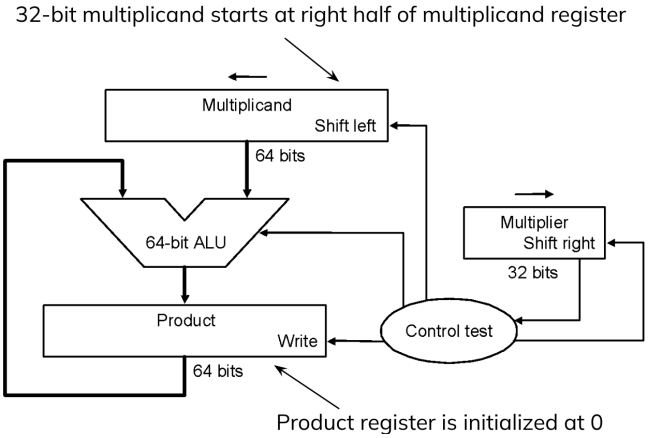
4.1 Multiplication

4.1.1 Version 1

#Question: 0010 * 0011

Itr	Step	Multiplier	Multiplicand	Product
0	Init	0011	0000 0010	0000 0000
1	1a	0011	0000 0010	0000 0010
	2	0011	0000 0100	0000 0010
	3	0001	0000 0100	0000 0010
2	1a	0001	0000 0100	0000 0110
	2	0001	0000 1000	0000 0110
	3	0000	0000 1000	0000 0110
3	1a	0000	0000 1000	0000 0110
	2	0000	0001 0000	0000 0110
	3	0000	0001 0000	0000 0110
4	1a	0000	0001 0000	0000 0110
	2	0000	0010 0000	0000 0110
	3	0000	0010 0000	0000 0110

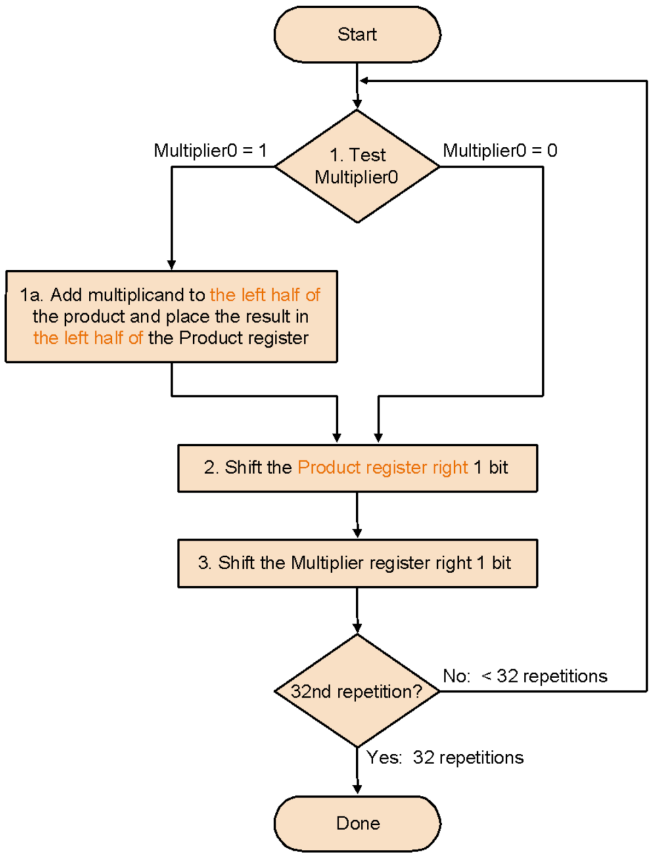


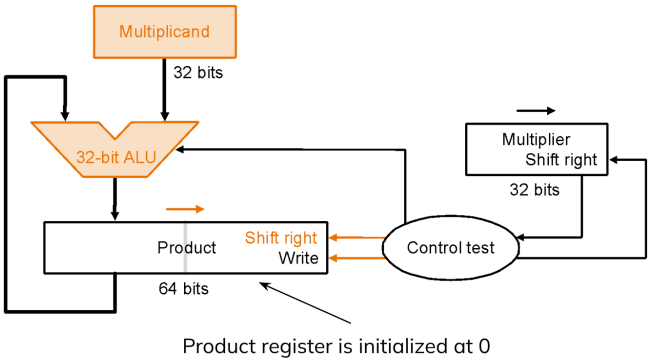


4.1.2 Version 2

***#Question*:** 0010 * 0011

Itr	Step	Multiplier	Multiplicand	Product
0	Init	0011	0010	0000 0000
1	1a	0011	0010	0010 0000
	2	0011	0010	0001 0000
	3	0001	0010	0001 0000
2	1a	0011	0010	0011 0000
	2	0011	0010	0001 1000
	3	0000	0010	0001 1000
3	1a	0010	0010	0001 1000
	2	0011	0010	0000 1100
	3	0000	0010	0000 1100
4	1a	0010	0010	0000 1100
	2	0011	0010	0000 0110
	3	0000	0010	0000 0110

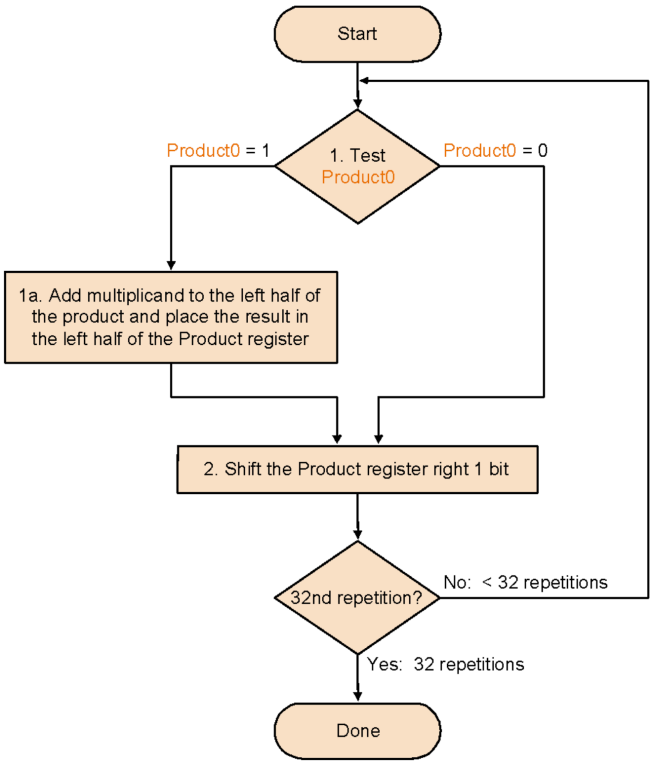


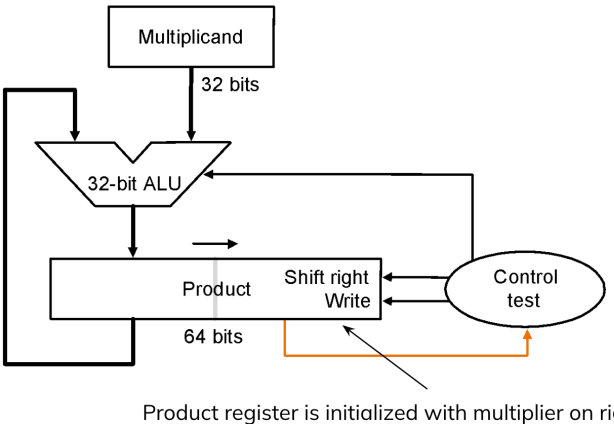


4.1.3 Version 3

***#Question*:** 0100 * 0101

Itr	Step	Multiplicand	Product
0	Init	0100	0000 0101
1	1a	0100	0100 0101
	2	0100	0010 0010
2	1a	0100	0010 0010
	2	0100	0001 0001
3	1a	0100	0101 0001
	2	0100	0010 1000
4	1a	0100	0010 1000
	2	0100	0001 0100





Product register is initialized with multiplier on right

4.1.4 Booth's Algorithm

#Question: 5 × -6

Operation	Accumulator (A)	Multiplier (Q)	Q_{-1}	Multiplicand (M)
Init	0 0 0 0	0 1 0 1	0	m = 1010 $-m$ = 0110
Q_0Q_{-1} = 10 Sub m *SAR*	0 1 1 0 0 1 1 0 0 0 1 1	0 1 0 1 0 0 1 0	1	
Q_0Q_{-1} = 01 Add m *SAR*	1 0 1 0 1 1 0 1 1 1 1 0	0 0 1 0 1 0 0 1	0	
Q_0Q_{-1} = 10 Sub m *SAR*	0 1 1 0 0 1 0 0 0 0 1 0	1 0 0 1 0 1 0 0	1	
Q_0Q_{-1} = 01 Add m *SAR*	1 0 1 0 1 1 0 0 1 1 1 0	0 1 0 0 0 0 1 0	0	

5 = 0101
-6 = 1010
6 = 0110

Operations:

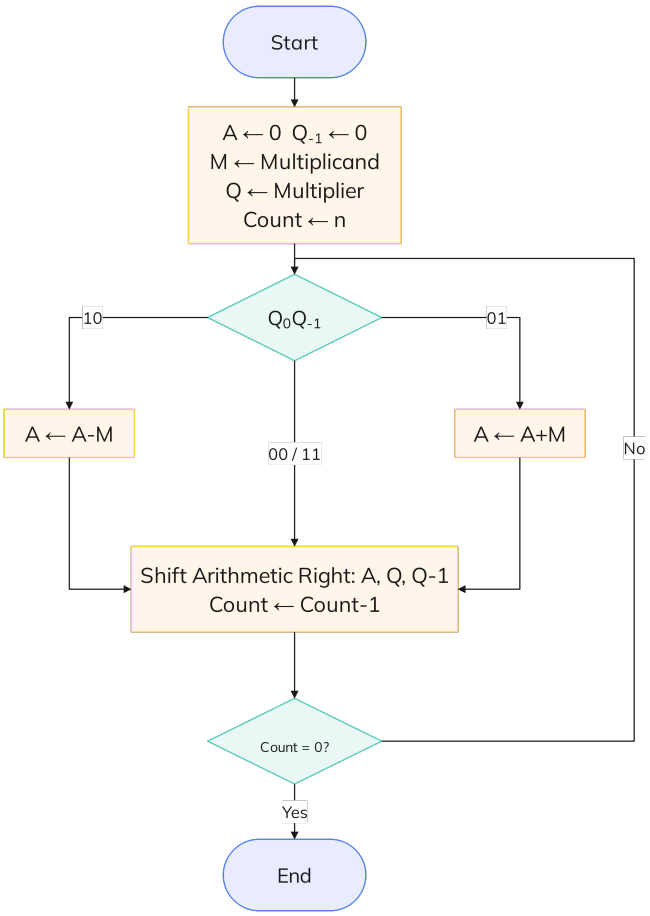
10 = Sub m

01 = Add m

00/11 = No operation

Answer:

∴ 5 × -6 = $(1111\ 0010)_2$ = $(-30)_{10}$



Chapter 5

Computer Performance

5.1 Terminology

- $\text{Performance}_X = \dfrac{1}{\text{Execution time}_X}$
- Cycle time (seconds per cycle)
- CPU time / Execution time (seconds per program)
- CPU clock cycles (cycles per program)
- Instruction count (instructions per program)
- Clock rate (cycles per second)
- CPI (cycles per instruction)
- MIPS (millions of instructions per second)
- ***Response time*** : The time from when a request (or task) is submitted to when the system starts responding is the response time. Lower the response time indicates faster service.
- ***Throughput*** : the total amount of work done in a given time.

$$\text{Clock Cycles (No of Pulse required)} = \text{CPU Time} \times \text{Clock Rate}$$

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \frac{\text{CPU Time} \times \text{Clock Rate}}{\text{Instruction Count}}$$

$$\text{Execution time} = \text{Clock cycles} \times \text{Clock cycle time}$$

$$\text{Execution time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

5.2 Numerical Problems

***#Question-1*:**
A designer building a new machine B, which must run a specific program in 6 seconds. The current machine, A, completes the same program in 10 seconds, using a 2.5 GHz clock rate. The designer plans to increase the clock rate to improve performance. However, this design change will cause Machine B to require 1.2 times as many clock cycles as Machine A for the same program. Based on this information, calculate the clock rate that Machine B must have.

***Solution*:**

Now,
For Machine A,
Clock Cycles $=$ CPU Time $\times$ Clock Rate
$= 10 \times 2.5 \times 10^9 = 2.5 \times 10^{10}$ cycles

For Machine B,
Clock Cycles $= 1.2 \times$ Machine A clock cycles
$= 1.2 \times 2.5 \times 10^{10} = 3 \times 10^{10}$ cycles

$$\therefore \text{CPU Time} = \frac{\text{Clock Cycles}}{\text{Clock Rate}}$$

$$\Rightarrow \text{Clock Rate} = \frac{\text{Clock Cycles}}{\text{CPU Time}}$$
$$= \frac{3 \times 10^{10}}{6}$$
$$= 5 \times 10^9 \text{Hz} = 5\text{GHz}$$

Given,

For Machine A,
Execution time = 10 seconds
Clock rate = 2.5 GHz = 2.5×10^9 Hz

For Machine B,
Execution time = 6 seconds
Clock cycles = 1.2 $\times$ Machine A clock cycles

***#Question-2*:**
Suppose Machine X needs 5 seconds to run a program and Y needs 3 seconds to run the same program and Clock rate is 3 GHz. Machine Y to require 2.5 times as many clock cycles as Machine X for the same program. What will be the clock rate?

***Solution*:**

Now,

For Machine X,
Clock cycles $=$ CPU time $\times$ Clock rate
$= 5 \times 3 \times 10^9 = 15 \times 10^9$ cycles

For Machine Y,
Clock cycles $= 2.5 \times$ Machine A clock cycles
$= 2.5 \times 15 \times 10^9 = 3.75 \times 10^{10}$ cycles

$$\therefore \text{CPU Time} = \frac{\text{Clock Cycles}}{\text{Clock Rate}}$$

$$\Rightarrow \text{Clock Rate} = \frac{\text{Clock Cycles}}{\text{CPU Time}}$$
$$= \frac{3.75 \times 10^{10}}{3}$$
$$= 1.25 \times 10^{10}\text{Hz} = 12.5\text{GHz}$$

Given,

Machine X,
Execution time = 5 seconds
Clock rate = 3 GHz = 3×10^9 Hz

Machine Y,
Execution time = 3 seconds

***#Question-3*:**

A compiler designer is trying to decide between two code sequences for a particular machine. Based on the hardware implementation, there are three different classes of instructions: Class A, Class B, and Class C, and they require 1, 2 and 3 cycles (respectively). The first code sequence has 5 instructions: 2 of A, 1 of B, and 2 of C The second sequence has 6 instructions: 4 of A, 1 of B, and 1 of C. Which sequence will be faster? How much? What is the CPI for each sequence?

***Solution*:**

Now,

For sequence-1,

$$\text{CPI} = \frac{\text{Totol CPU Cycles}}{\text{Instruction Count}}$$
$$= \frac{(2 \times 1) + (1 \times 2) + (2 \times 3)}{5}$$
$$= \frac{10}{5} = 2 \text{ cycles per instruction}$$

For sequence-2,

$$\text{CPI} = \frac{\text{Totol CPU Cycles}}{\text{Instruction Count}}$$
$$= \frac{(4 \times 1) + (1 \times 2) + (1 \times 3)}{6}$$
$$= \frac{9}{6} = 1.5 \text{ cycles per instruction}$$

Now,

Let, Clock cycle time $= 1ns$

$\therefore \text{ET}_1 = \text{Total cycles} \times \text{Clock cycle time} = 10 \times 1 \quad \ldots \quad (1)$

$\therefore \text{ET}_2 = \text{Total cycles} \times \text{Clock cycle time} = 9 \times 1 \quad \ldots \quad (2)$

$$(2) \div (1) \Rightarrow \frac{\text{ET}_2}{\text{ET}_1} = \frac{9}{10} = 0.9$$

$\therefore$ Sequence 2 is 0.9 times faster than sequence 1.

Given,

Cycles required for,
Class A = 1
Class B = 2
Class C = 3

For sequence-1,
Instructions: 2 of A, 1 of B, and 2 of C = 5 cycles

For sequence-2,
Instructions: 4 of A, 1 of B, and 1 of C = 6 cycles

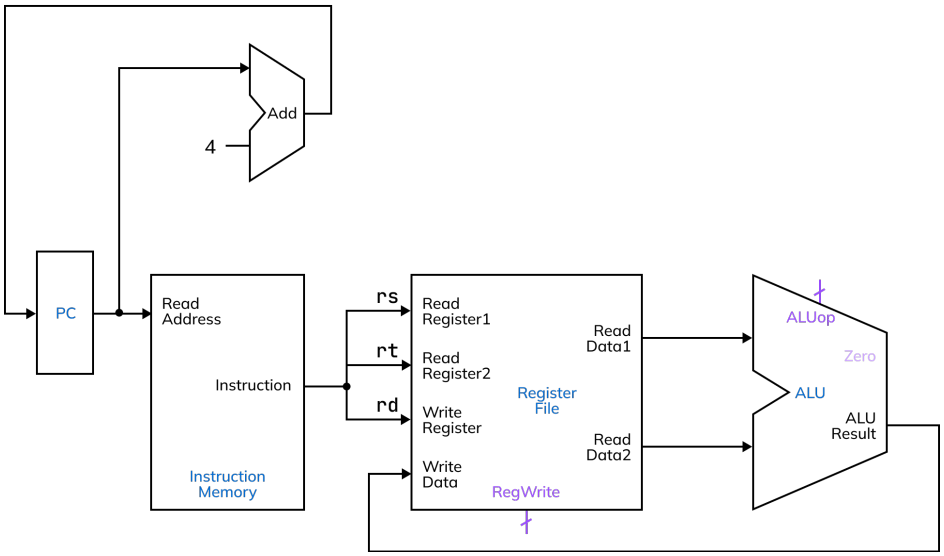
Chapter 6

Datapath

6.1 R Type Instruction

Question: Draw and explain the datapath for `add $rd, $rs, $rt` with control signals.

Solution:



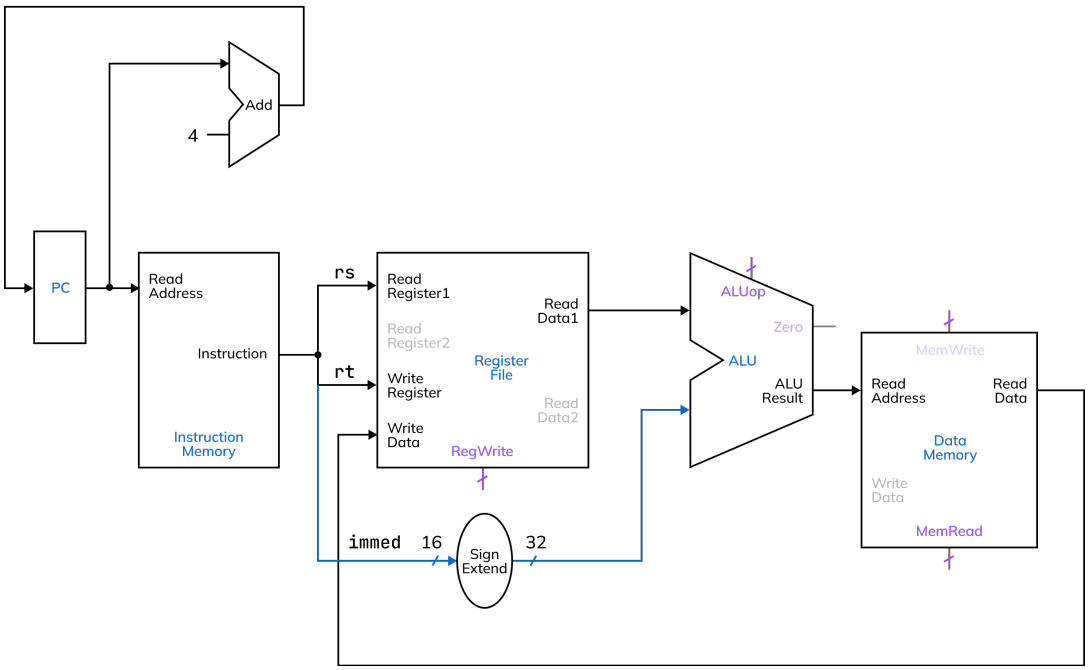
1. Grab instruction address from PC.
2. Fetch instruction from instruction memory.
3. Decode instruction.
4. Pass `rs`, `rt`, and `rd` into read register and write register arguments.
5. Retrieve data from read register 1 and read register 2 (`rs` and `rt`).
6. Pass contents of `rs` and `rt` into the ALU as operands of the operation to be performed.
7. Retrieve result of operation performed by ALU and pass back as the write data argument of the register file (with the `RegWrite` bit set).
8. Add 4 bytes to the PC value to obtain the word-aligned address of the next instruction.

6.2 I Type Instruction

6.2.1 Load Instruction

Question: Draw and explain the datapath for `lw $rt, immed($rs)` with control signals.

Solution:

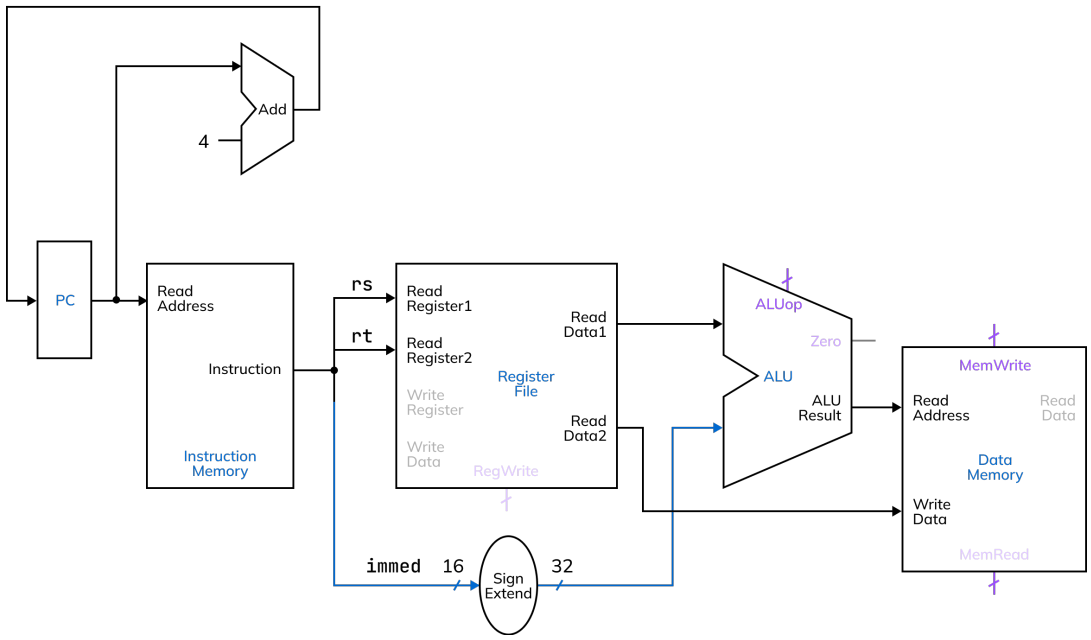


1. Grab instruction address from PC.
2. Fetch instruction from instruction memory.
3. Decode instruction and extract `rs`, `rt`, and `immediate` fields.
4. Use `rs` to access base address from register file (read register 1).
5. Sign-extend the 16-bit immediate value to 32 bits.
6. Add the sign-extended immediate to the value from `rs` using the ALU to calculate the address.
7. Retrieve the calculated effective memory address from ALU and pass back as the read address argument of the data memory (with `MemRead` enabled).
8. Retrieve the data from data memory and pass back as the write data argument of the register file (with the `RegWrite` bit set).
9. Add 4 bytes to the PC to obtain the word-aligned address of the next instruction.

6.2.2 Store Instruction

***Question*:** Draw and explain the datapath for `sw $rt, immed($rs)` with control signals.

***Solution*:**

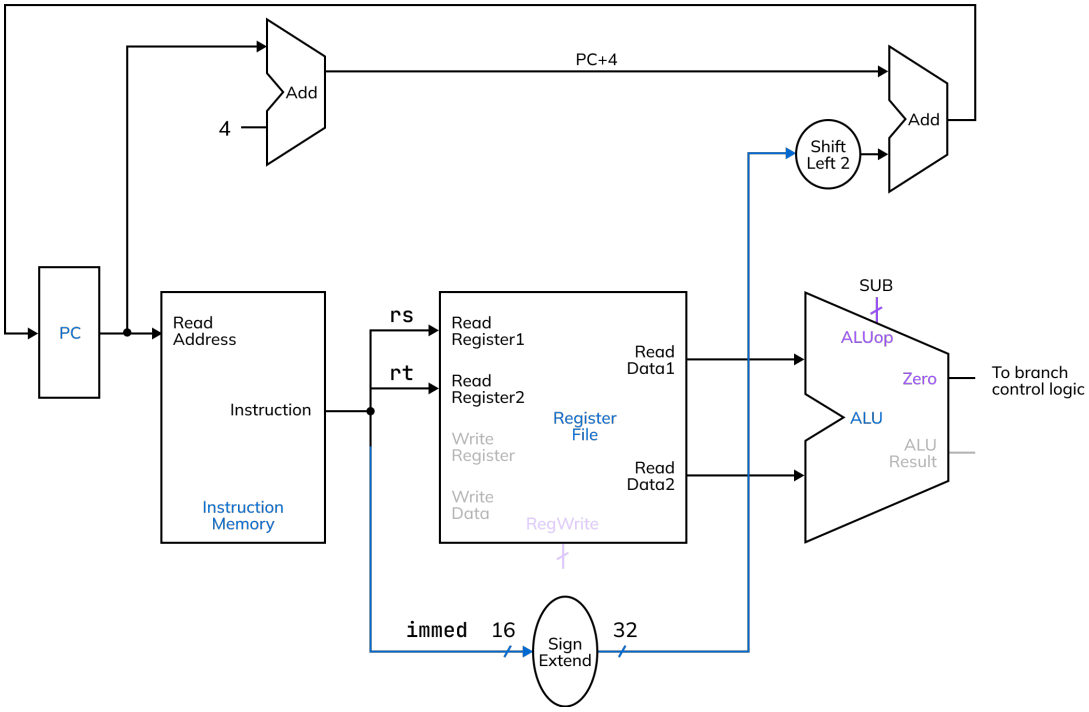


1. Grab instruction address from PC.
2. Fetch instruction from instruction memory.
3. Decode instruction and extract `rs`, `rt`, and `immediate` fields.
4. Use `rs` to access base address from register file (read register 1).
5. Use `rt` to access the value that needs to be stored (read register 2).
6. Sign-extend the 16-bit immediate value to 32 bits.
7. Add the sign-extended immediate to the value from `rs` using the ALU to calculate the effective memory address.
8. Pass the ALU result as the write address to the data memory (with `MemWrite` enabled).
9. Pass the value from `rt` as the write data to memory.
10. Add 4 bytes to the PC to obtain the word-aligned address of the next instruction.

6.2.3 Branch Instruction

***Question*:** Draw and explain the datapath for `beq $rs, $rt, target` with control signals.

***Solution*:**



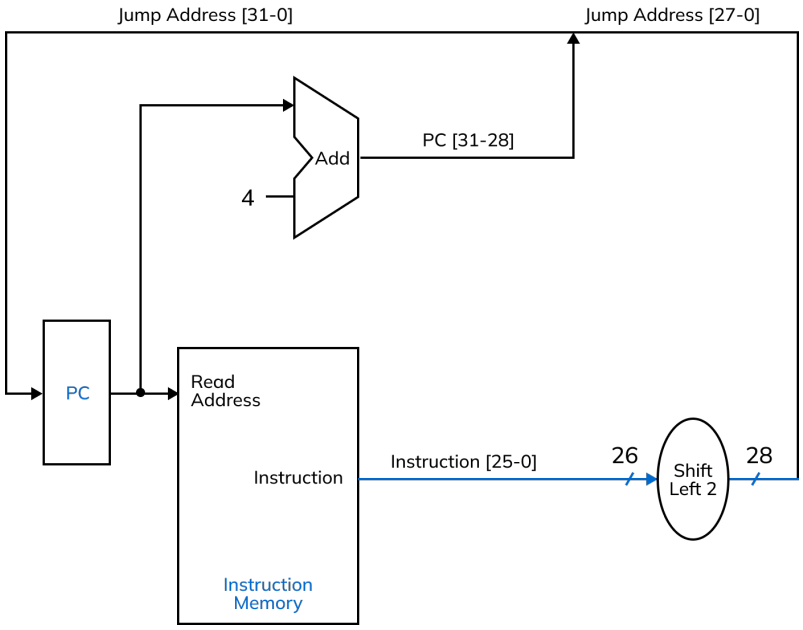
1. Grab instruction address from PC.
2. Fetch instruction from instruction memory.
3. Decode instruction and extract `rs`, `rt`, and `immediate` fields.
4. Use `rs` and `rt` to read values from register file (read register 1 and read register 2).
5. Pass both register values into ALU to perform subtraction:
   - ALU does: ReadData1 - ReadData2
6. Check if ALU result is zero (i.e., `$t1` = `$t2`) - set the `Zero` signal.
7. Sign-extend the 16-bit immediate value to 32 bits.
8. Shift the sign-extended immediate left by 2 bits to get word-aligned offset.
9. Add this shifted offset to PC + 4 to calculate branch target address.
10. If `Zero` = 1 and `Branch` = 1, update PC to branch target address.
11. If not, PC is updated to PC + 4 to move to next instruction normally.

6.3 J Type Instruction

6.3.1 Jump Instruction

***Question*:** Draw and explain the datapath for `j target` with control signals.

***Solution*:**



1. Grab instruction address from PC.
2. Fetch instruction from instruction memory.
3. Extract the 26-bit target address field from the instruction (bits [25-0]).
4. Shift the 26-bit target address left by 2 bits to make it word-aligned (now 28 bits).
5. Take the upper 4 bits (bits [31-28]) from PC + 4 to preserve current PC region.
6. Concatenate upper 4 bits of PC+4 with the 28-bit shifted address forms the full 32-bit jump address.
7. Load this new address into the PC execution jumps to that address.
8. No register read/write or ALU operation is needed in this instruction.